

miniFlink

Декларативная потоковая обработка данных на OCaml — учебная реализация ключевых концепций Apache Flink.

miniFlink выражает семантику потоковой обработки максимально прозрачно: весь конвейер мониторинга читается как спецификация, без явного `key_by`, без аннотаций типов, без ручных проверок на `null`.

```
let pipeline source =
  source
  |> Mf_event.with_watermarks ~latency:(seconds 3)
  |> Pipe.enrich (module Telemetry)
    ~from:devices
    ~merge:(fun t dev -> { t with device = dev })
  |> Pipe.window (module Telemetry) (Pipe.tumbling (seconds 30))
  |> Pipe.aggregate Rules.compute
  |> Pipe.flat_map (Rules.check Rules.fleet)
  |> Pipe.dedup (module Alert)
    ~rule:(fun a -> a.rule)
    ~cooldown:(minutes 5)
```

Обзор архитектуры

Проект разбит на ортогональные слои. Каждый слой зависит только от нижележащих; ядро никогда не импортирует слои надёжности или параллелизма.

Ядро

Никогда не меняется, без внешних зависимостей.

- `Stream` — pull-based поток `unit -> 'a option`. Стек вызовов = граф операторов.
- `Mf_event` — событие `Data | Watermark | Retract`. Watermark-стратегия.
- `Time` — единицы времени: `seconds`, `minutes`, `hours`.
- `Keyed` — type class `KEYED`: ключ группировки описывается один раз в типе.
- `Pipe` — операторы: `enrich`, `window`, `aggregate`, `dedup`, `flat_map`.
- `Table` — таблицы `of_list`, `of_stream`, `of_stream_ttl` для join.
- `Codec` — абстракция сериализации JSON / Protobuf.

Домен

- `Domain` — доменные типы с `@@deriving yojson, show`.
- `Rules` — бизнес-логика как чистые функции.

Параллелизм

Шардирование по ключу: `hash(key) mod N` направляет событие воркеру.

- `Channel` — канал между операторами с backpressure (bounded).
- `Parallel` — data-parallel выполнение. `Thread` (OCaml 4) / `Domain` (OCaml 5).
- `Checkpoint_parallel` — exactly-once: barrier-инжекция + per-worker snapshot.

Production-слои

Каждый: контракт (`.mli`) + `_noop` (нулевой overhead) + `_default` (production).

- `Dlq` — dead letter queue: ошибки decode не роняют конвейер.
- `Shutdown` — graceful shutdown: `SIGTERM` → `drain` → `exit`.
- `Metrics` — Prometheus-совместимые метрики + HTTP endpoint.
- `Barrier` — координатор Chandy-Lamport для распределённого снапшота.
- `State_backend` — хранилище стейта: `memory` / `RocksDB` (через C FFI).
- `Schema` — эволюция схемы.
- `Harness` — детерминированный тестовый фреймворк.
- `Runtime` — сборка всех слоёв по режиму (`Noop | Log | Parallel | Prod`).

С чего начать

Тutorial с пошаговым построением конвейера: [tutorial](#).

Запуск

```
dune build          # сборка (channel/parallel выбираются по версии OCaml)
dune exec bin/main.exe  # демо
dune test           # 13 тест-скит
dune exec bench/soak.exe -- 2000000 # soak: проверка отсутствия утечек
```

Гарантии и границы

Реализовано: event time, watermarks (с периодической эмиссией и финальным flush), tumbling/sliding окна, stateful-операторы, exactly-once стейта в параллельном режиме (barrier + recovery), table/join с TTL-eviction, retractions для поздних данных, персистентный RocksDB-стейт.

Сознательно вне области (*non-goals*): распределённое выполнение, remote shuffle, end-to-end exactly-once через внешние source/sink, incremental checkpointing. «mini» в названии честное — проект про *семантику*, не про распределённую инфраструктуру.

Module Stream

Pull-based ленивый поток.

Поток — это функция `unit -> 'a option`: каждый вызов возвращает следующий элемент, `None` означает конец. Стек вызовов операторов *и есть* граф обработки — никаких колбэков и промежуточных буферов (кроме `flat_map`, где буфер необходим).

```
type 'a t = unit -> 'a option
```

Тип потока элементов `'a`. Поток ленивый и одноразовый: элементы вычисляются по требованию, повторный проход требует нового потока.

```
val of_list : 'a list -> 'a t
```

Поток из списка (в исходном порядке).

```
val empty : 'a t
```

Пустой поток (сразу `None`).

```
val map : ('a -> 'b) -> 'a t -> 'b t
```

`map f s` применяет `f` к каждому элементу лениво.

```
val filter : ('a -> bool) -> 'a t -> 'a t
```

`filter p s` пропускает только элементы, удовлетворяющие `p`.

```
val flat_map : ('a -> 'b list) -> 'a t -> 'b t
```

`flat_map f s` заменяет каждый элемент списком `f x`, уплотняя результат. Единственный оператор с внутренним буфером.

```
val iter : ('a -> unit) -> 'a t -> unit
```

`iter f s` вызывает `f` на каждом элементе ради эффекта, проходя поток до конца.

```
val fold : ('a -> 'b -> 'a) -> 'a -> 'b t -> 'a
```

`fold f z s` сворачивает поток слева: `f (... (f z x1) ...) xn`.

```
val to_list : 'a t -> 'a list
```

Собрать поток в список (в исходном порядке). Проходит поток до конца.

Module Time

Единицы времени. Внутреннее представление — миллисекунды (`int`), все события и `watermarks` используют эту шкалу. Конструкторы ниже делают код читаемым: `seconds 30` вместо `30_000`.

```
type t = int
```

Время в миллисекундах.

```
val ms : int -> t
```

Миллисекунды как есть.

```
val seconds : int -> t
```

Секунды → миллисекунды.

```
val minutes : int -> t
```

Минуты → миллисекунды.

```
val hours : int -> t
```

Часы → миллисекунды.

Module Mf_event

События потока: данные, watermark, retract.

Каждое значение несёт свой event-time. Watermark — граница, после которой события раньше неё уже не придут (позволяет закрывать окна детерминированно). Retract отменяет ранее выданный результат (для поздних данных).

```
type 'a t =  
  | Data of 'a * Time.t  
  | Watermark of Time.t  
  | Retract of 'a * Time.t
```

значение + его event-time

граница: события раньше уже пришли

отмена ранее выданного результата

Событие с полезной нагрузкой 'a. Конструкторы открыты — операторы сопоставляются с ними напрямую.

```
val data : 'a -> Time.t -> 'a t
```

data v ts — событие-данные со значением v на event-time ts.

```
val retract : 'a -> Time.t -> 'a t
```

retract v ts — отмена ранее выданного v.

```
val wm : Time.t -> 'a t
```

wm ts — watermark на момент ts.

```
val value : 'a t -> 'a option
```

Значение события (Data/Retract) или None для watermark.

```
val ts : 'a t -> Time.t
```

Event-time события (для любого вида).

```
val is_data : 'a t -> bool
```

Является ли событие Data.

```
val map_value : ('a -> 'b) -> 'a t -> 'b t
```

Применить функцию к значению, сохранив вид и время; watermark — без изменений.

```
val with_watermarks_ext :  
  latency:Time.t ->  
  ?interval:Time.t ->  
  'a t Stream.t ->  
  'a t Stream.t
```

with_watermarks_ext ~latency ?interval src вставляет watermarks max_seen - latency в поток.

- ~latency — допуск на опоздание (насколько событие может прийти позже своего времени).
- ~interval — минимальный сдвиг watermark перед новой эмиссией (по event-time); 0 = на каждый новый

максимум, больше = реже. На конце потока эмитит финальный watermark, закрывающий все окна.

```
val with_watermarks : latency:Time.t -> 'a t Stream.t -> 'a t Stream.t
```

`with_watermarks ~latency src — with_watermarks_ext c ~interval:0` (watermark на каждый новый максимум).

```
val with_idle_watermarks :  
  latency:Time.t ->  
  idle_ms:int ->  
  idle_advance:Time.t ->  
  now_ms:(unit -> int) ->  
  sleep_ms:(int -> unit) ->  
  (unit -> 'a t option option) ->  
  'a t Stream.t
```

Idle watermark: продвигает watermark по *wall-clock* когда источник замолчал, чтобы окна не висели в тишине.

Источник *неблокирующий*: `poll : unit -> 'a t option option` — `None` (поток окончен), `Some None` (данных пока нет), `Some (Some ev)` (событие). При тишине дольше `~idle_ms` эмитит watermark, продвинутый на `~idle_advance`. `~now_ms/~sleep_ms` вынесены параметрами для тестируемости (управляемые часы вместо реальных). Таймер живёт здесь, в слое источника — чистое event-time ядро не затрагивается.

```
val pp_ts : Time.t -> string
```

Форматировать время как «1.234s» (для отладочного вывода).

Module Keyed

Type class «имеет ключ».

Любой тип, реализующий `S`, работает с `window`, `enrich`, `dedup` без явного `~key` в каждом операторе — ключ описывается один раз. Это убирает повторение `(fun t -> t.device_id)` во всех вызовах.

```
module type S = sig ... end
```

Тип с ключом группировки.

```
module Make (T : sig ... end) : S with type t = T.t
```

Создать keyed-модуль из типа и функции ключа:

Module Table

Таблица как функция ключ → значение `option` — для `enrich` (обогащение потока справочными данными, left join).

Три конструктора: статический список, живой `hashtbl`, и `snapshot` из потока (последнее значение по ключу).

```
type ('k, 'v) t = 'k -> 'v option
```

Таблица: поиск значения `'v` по ключу `'k`.

```
val of_list : ('k * 'v) list -> ('k, 'v) t
```

Статическая таблица из списка пар. Загружается один раз; для справочников, не меняющихся во время работы.

```
val of_hashtbl : ('k, 'v) Stdlib.Hashtbl.t -> ('k, 'v) t
```

Таблица поверх изменяемого `Hashtbl` — отражает текущее содержимое при каждом поиске.

```
val of_stream : key:('a -> 'k) -> 'a Mf_event.t Stream.t -> ('k, 'a) t
```

Живой `snapshot` из потока: последнее `Data`-значение по ключу.

Размер ограничен числом *уникальных* ключей (replace, не add). Утечка возможна только при неограниченно растущем пространстве ключей — для такого случая см. `of_stream_ttl`.

Замечание: при поиске источник вычитывается до `None`. Для живого канала `None` = «пока пусто» (ок); для конечного `Stream.of_list` первый поиск осушит источник — для статики берите `of_list`.

```
val of_stream_ttl :  
  key:('a -> 'k) ->  
  ttl:int ->  
  'a Mf_event.t Stream.t ->  
  ('k, 'a) t
```

Как `of_stream`, но записи старше `ttl` (относительно последнего виденного event-time) удаляются — ограничивает размер при растущем пространстве ключей.

Module Pipe

Операторы конвейера.

Каждый оператор берёт поток событий (`Mf_event.t Stream.t`) и возвращает поток. Операторы с группировкой (`window`, `enrich`, `dedup`, `count/session/global` окна) получают ключ через первый аргумент-модуль `Keyed.S` — в пользовательском коде нет повторов извлечения ключа.

Watermark и Retract проходят через операторы прозрачно (где это осмысленно), сохраняя event-time семантику.

Базовые операторы (по значению)

```
val map : ('a -> 'b) -> 'a Mf_event.t Stream.t -> 'b Mf_event.t Stream.t
```

`map f` применяет `f` к значению каждого `Data/Retract`; watermark без изменений.

```
val filter : ('a -> bool) -> 'a Mf_event.t Stream.t -> 'a Mf_event.t Stream.t
```

`filter p` оставляет `Data`, удовлетворяющие `p`; watermark и retract проходят.

```
val flat_map :  
  ('a -> 'b list) ->  
  'a Mf_event.t Stream.t ->  
  'b Mf_event.t Stream.t
```

`flat_map f` заменяет каждое значение списком `f v`, сохраняя event-time и вид события.

Обогащение

```
val enrich :  
  (module Keyed.S with type t = 'a) ->  
  from:(string, 'b) Table.t ->  
  merge:('a -> 'b option -> 'a) ->  
  'a Mf_event.t Stream.t ->  
  'a Mf_event.t Stream.t
```

`enrich (module K) ~from ~merge upstream` для каждого события ищет в таблице `from` значение по ключу `K.key` и сливает его в событие через `merge` (left join: `None` если ключа нет).

Окна по времени

```
type win_spec
```

Спецификация временного окна.

```
val tumbling : Time.t -> win_spec
```

Неперекрывающиеся окна размера `size` (`> 0`).

Raises `Invalid_argument`

при `size <= 0`.

```
val sliding : Time.t -> Time.t -> win_spec
```

Перекрывающиеся окна размера `size` с шагом `step` (при `step < size` окна перекрываются).

Raises `Invalid_argument`

при `size <= 0` или `step <= 0`.

```
val assign : win_spec -> Time.t -> (Time.t * Time.t) list
```

Какие окна назначаются событию с временем `ts` по спецификации. Низкоуровневая деталь (используется внутри `window`); экспонирована для модульного тестирования логики назначения окон.

```
val window :  
  (module Keyed.S with type t = 'a) ->  
  ?latency:Time.t ->  
  ?allowed_lateness:Time.t ->  
  win_spec ->  
  'a Mf_event.t Stream.t ->  
  (string * 'a list) Mf_event.t Stream.t
```

`window (module K) ?latency ?allowed_lateness spec upstream` группирует события по ключу `K.key` и временным окнам `spec`. Окно закрывается (эмитит `(key, values)`) когда watermark проходит его конец. `?allowed_lateness` продлевает приём опоздавших после закрытия.

```
val aggregate :  
  (string -> 'a list -> 'b) ->  
  (string * 'a list) Mf_event.t Stream.t ->  
  'b Mf_event.t Stream.t
```

`aggregate f` сворачивает каждое окно `(key, values)` в результат `f key values`.

Count-окна (по количеству событий, без watermarks)

```
type count_spec
```

Спецификация count-окна.

```
val count_tumbling : int -> count_spec
```

Окно каждые `n` событий (`> 0`).

```
val count_sliding : int -> int -> count_spec
```

Окно из `n` событий с шагом `step` (оба `> 0`).

```
val count_window :  
  (module Keyed.S with type t = 'a) ->  
  count_spec ->  
  'a Mf_event.t Stream.t ->  
  (string * 'a list) Mf_event.t Stream.t
```

`count_window (module K) spec upstream` группирует по ключу и числу событий, без watermarks. Неполные хвосты не эмитятся.

Raises `Invalid_argument`

при недопустимых параметрах.

Session-окна (динамические границы со слиянием)

```
val session_window :  
  (module Keyed.S with type t = 'a) ->  
  gap:Time.t ->  
  'a Mf_event.t Stream.t ->  
  (string * 'a list) Mf_event.t Stream.t
```

`session_window (module K) ~gap upstream` группирует события в сессии — периоды активности, разделённые

паузами больше `gap`. Сессии *сливаются* когда событие перекрывает разрыв между ними. Закрытие по watermark (`last + gap`) или на конце потока.

Raises `Invalid_argument`

при `gap <= 0`.

Global-окно с триггерами

```
type trigger_action =  
  | Continue  


копить дальше

  
  | Fire  


эмитить, оставив накопленное (накопительно)

  
  | FireAndPurge  


эмитить и сбросить буфер


```

Решение триггера.

```
type 'a trigger = count:int -> last:'a -> trigger_action
```

Триггер: по числу накопленных и последнему значению — решение.

```
val trigger_count : int -> 'a trigger
```

Триггер «каждые `n` событий» (с `purge`).

```
val trigger_on_value : ('a -> bool) -> 'a trigger
```

Триггер по предикату значения (ранняя эмиссия).

```
val global_window :  
  (module Keyed.S with type t = 'a) ->  
  trigger:'a trigger ->  
  'a Mf_event.t Stream.t ->  
  (string * 'a list) Mf_event.t Stream.t
```

`global_window (module K) ~trigger upstream` — одно окно на ключ, эмиссия по `trigger`. Отделяет группировку от политики «когда фаерить».

Состояние и дедупликация

```
val stateful :  
  init:'s ->  
  f:( 's -> 'a Mf_event.t -> 's * 'b Mf_event.t list) ->  
  'a Mf_event.t Stream.t ->  
  'b Mf_event.t Stream.t
```

`stateful ~init ~f upstream` — оператор с состоянием: `f state event` возвращает новое состояние и список выходных событий. Watermark проходит прозрачно.

```
val dedup :  
  (module Keyed.S with type t = 'a) ->  
  rule:( 'a -> string) ->  
  cooldown:Time.t ->  
  'a Mf_event.t Stream.t ->
```

```
'a Mf_event.t Stream.t
```

`dedup (module K) ~rule ~cooldown upstream` подавляет повторы с одинаковым (`K.key`, `rule`) в пределах `cooldown`. Состояние ограничено: при watermark записи старше `wm - cooldown` удаляются.

Терминальные

```
val sink : ('a -> unit) -> 'a Mf_event.t Stream.t -> unit
```

`sink f stream` вызывает `f` на каждом `Data`-значении (ради эффекта), проходя поток до конца.

```
val collect : 'a Mf_event.t Stream.t -> 'a list
```

Собрать `Data`-значения потока в список.

Module Codec

Кодеки (де)сериализации значений в/из `bytes`.

`decode` возвращает `result` — битый ввод даёт `Error`, не исключение. Смена формата (JSON ↔ Protobuf) — замена кодека одной строкой, сам пайплайн не меняется.

```
type 'a t = {  
  encode : 'a -> bytes;  
  decode : bytes -> ('a, string) Stdlib.result;  
  name : string;  


имя формата (для логов/метрик)

  
}
```

Кодек значений типа `'a`.

```
val json :  
  encode:('a -> Yojson.Safe.t) ->  
  decode:(Yojson.Safe.t -> ('a, string) Stdlib.result) ->  
  'a t
```

JSON-кодек поверх Yojson. `encode/decode` работают с `Yojson.Safe.t`; ошибки парсинга JSON оборачиваются в `Error`.

```
val protobuf : encode:('a -> bytes) -> decode:(bytes -> 'a) -> 'a t
```

Protobuf-подобный кодек поверх произвольных `bytes`. Исключение из `decode` оборачивается в `Error`.

Module Domain

```
type position = {  
  lat : float;  
  lon : float;  
}
```

```
val position_to_yojson : position -> Yojson.Safe.t
```

```
val position_of_yojson :  
  Yojson.Safe.t ->  
  position Ppx_deriving_yojson_runtime.error_or
```

```
val pp_position :  
  Ppx_deriving_runtime.Format.formatter ->  
  position ->  
  Ppx_deriving_runtime.unit
```

```
val show_position : position -> Ppx_deriving_runtime.string
```

```
type device_info = {  
  owner : string;  
  max_speed : float;  
  zone : string;  
}
```

```
val device_info_to_yojson : device_info -> Yojson.Safe.t
```

```
val device_info_of_yojson :  
  Yojson.Safe.t ->  
  device_info Ppx_deriving_yojson_runtime.error_or
```

```
val pp_device_info :  
  Ppx_deriving_runtime.Format.formatter ->  
  device_info ->  
  Ppx_deriving_runtime.unit
```

```
val show_device_info : device_info -> Ppx_deriving_runtime.string
```

```
type telemetry = {  
  device_id : string;  
  speed_kmh : float;  
  fuel_pct : float;  
  position : position;  
  ts : int;  
  device : device_info option;  
}
```

```
val telemetry_to_yojson : telemetry -> Yojson.Safe.t
```

```
val telemetry_of_yojson :
```

```
Yojson.Safe.t ->  
telemetry Ppx_deriving_yojson_runtime.error_or
```

```
val pp_telemetry :  
  Ppx_deriving_runtime.Format.formatter ->  
  telemetry ->  
  Ppx_deriving_runtime.unit
```

```
val show_telemetry : telemetry -> Ppx_deriving_runtime.string
```

```
module Telemetry : sig ... end
```

```
type stats = {  
  device_id : string;  
  max_speed : float;  
  avg_speed : float;  
  min_fuel : float;  
  count : int;  
  device : device_info option;  
}
```

```
val stats_to_yojson : stats -> Yojson.Safe.t
```

```
val stats_of_yojson :  
  Yojson.Safe.t ->  
  stats Ppx_deriving_yojson_runtime.error_or
```

```
val pp_stats :  
  Ppx_deriving_runtime.Format.formatter ->  
  stats ->  
  Ppx_deriving_runtime.unit
```

```
val show_stats : stats -> Ppx_deriving_runtime.string
```

```
module Stats : sig ... end
```

```
type severity =  
  | Critical  
  | Warning  
  | Info
```

```
val severity_to_yojson : severity -> Yojson.Safe.t
```

```
val severity_of_yojson :  
  Yojson.Safe.t ->  
  severity Ppx_deriving_yojson_runtime.error_or
```

```
val pp_severity :  
  Ppx_deriving_runtime.Format.formatter ->  
  severity ->  
  Ppx_deriving_runtime.unit
```

```
val show_severity : severity -> Ppx_deriving_runtime.string
```

```
type alert = {  
  id : string;  
  device_id : string;  
  rule : string;  
  severity : severity;  
  message : string;  
  ts : int;  
}
```

```
val alert_to_yojson : alert -> Yojson.Safe.t
```

```
val alert_of_yojson :  
  Yojson.Safe.t ->  
  alert Ppx_deriving_yojson_runtime.error_or
```

```
val _ : Yojson.Safe.t -> alert Ppx_deriving_yojson_runtime.error_or
```

```
val pp_alert :  
  Ppx_deriving_runtime.Format.formatter ->  
  alert ->  
  Ppx_deriving_runtime.unit
```

```
val show_alert : alert -> Ppx_deriving_runtime.string
```

```
module Alert : sig ... end
```

```
val telemetry_json : telemetry Codec.t
```

```
val alert_json : alert Codec.t
```

Module Rules

```
val compute : string -> Domain.telemetry list -> Domain.stats
```

```
val alert :  
  rule:string ->  
  severity:Domain.severity ->  
  msg:(Domain.stats -> string) ->  
  Domain.stats ->  
  Domain.alert
```

```
val speed_limit : Domain.stats -> Domain.alert list
```

```
val fuel_level : Domain.stats -> Domain.alert list
```

```
val fleet : (Domain.stats -> Domain.alert list) list
```

```
val check :  
  (Domain.stats -> Domain.alert list) list ->  
  Domain.stats ->  
  Domain.alert list
```

Module Channel

Канал между операторами с backpressure.

Используется для передачи событий от dispatcher к воркерам в параллельном режиме. Bounded-канал даёт backpressure автоматически: producer блокируется когда канал полон, consumer — когда пуст.

Две реализации выбираются по версии OCaml (через dune-правило):

- **OCaml 4:** Mutex + Condition (Thread);
- **OCaml 5:** Atomic lock-free SPSC ring buffer (Domain).

Sentinel `None` из `pop` означает что канал закрыт и пуст.

```
type 'a t
```

Непрозрачный тип канала элементов `'a`.

```
val make_unbounded : unit -> 'a t
```

Создать неограниченный канал (без backpressure).

```
val make_bounded : int -> 'a t
```

Создать ограниченный канал заданной ёмкости (с backpressure).

```
val push : 'a t -> 'a -> unit
```

Записать значение. Блокирует если канал bounded и полон.

```
val try_push : 'a t -> 'a -> bool
```

Как `push`, но возвращает `false` если канал закрыт (например, consumer-воркер упал), вместо блокировки навсегда. Предотвращает deadlock при падении воркера.

```
val pop : 'a t -> 'a option
```

Прочитать значение. Блокирует если канал пуст и не закрыт. `None` — канал закрыт и пуст.

```
val try_pop : 'a t -> 'a option
```

Неблокирующее чтение. `None` — нет данных *сейчас*.

```
val close : 'a t -> unit
```

Закрыть канал. После этого `pop` вернёт оставшиеся элементы, затем `None`.

```
val length : 'a t -> int
```

Текущее число элементов в канале.

```
val to_stream : 'a t -> 'a Stream.t
```

Представить канал как pull-поток `Stream.t`.

Module Parallel

```
type worker_stats = {  
  mutable processed : int;  
  mutable emitted : int;  
  mutable wm_seen : int;  
}
```

```
val make_stats : unit -> worker_stats
```

```
val hash_key : string -> int -> int
```

```
val run_parallel_simple :  
  ?sink_factory:(int -> 'b -> unit) ->  
  ?on_queue_depth:(int array -> unit) ->  
  workers:int ->  
  capacity:int ->  
  key_of:('a -> string) ->  
  pipeline:('a Mf_event.t Stream.t -> 'b Mf_event.t Stream.t) ->  
  source:'a Mf_event.t Stream.t ->  
  sink:('b -> unit) ->  
  unit ->  
  unit
```

Data-parallel pipeline. Шардирует source по hash(key_of) на N воркеров. Каждый воркер запускает свою копию pipeline. Sink защищён мьютексом (v4) или вызывается per-domain (v5).

`?on_queue_depth` — опциональный хук наблюдаемости: dispatcher периодически вызывает его с массивом текущих глубин входных каналов (по воркерам). Раннее предупреждение о backpressure: растущая глубина = воркеры не успевают. Библиотека только *сообщает* глубину — что делать (gauge, лог, алерт) решает вызывающий.

Module Checkpoint_parallel

```
type epoch = int
```

```
type offset = int
```

Позиция чтения в источнике. Для Kafka — offset партии, для списка — индекс, для файла — байтовое смещение.

```
type 'a seekable_source = {  
  pull : unit -> 'a Mf_event.t option;  
  position : unit -> offset;  
  seek : offset -> unit;  
}
```

Адресуемый (seekable) источник: помимо чтения событий умеет сообщать текущую позицию и перематываться на сохранённую. Это то что делает возможным exactly-once на входной стороне — после сбоя мы перематываем источник на offset последнего закомиченного checkpoint и переобрабатываем без пропусков.

```
val seekable_of_list : 'a Mf_event.t list -> 'a0 seekable_source
```

Обернуть обычный список в seekable source (для тестов и in-memory).

```
type 'a msg =  
  | Event of 'a Mf_event.t  
  | Barrier of epoch
```

```
type worker_snapshot = {  
  worker : int;  
  epoch : epoch;  
  state : bytes;  
  processed : int;  
}
```

```
type checkpoint = {  
  cp_epoch : epoch;  
  cp_offset : offset;  
  cp_snapshots : worker_snapshot array;  
}
```

Полная запись checkpoint: epoch, позиция источника на момент barrier, и снапшоты всех воркеров. Source offset — ключевое добавление: без него восстановление не знает откуда читать.

```
type checkpoint_store = {  
  mutable committed : checkpoint list;  
  cs_mu : Mutex.t;  
  cs_persist : (checkpoint -> unit) option;  
}
```

```
val make_store : ?persist:(checkpoint -> unit) -> unit -> checkpoint_store
```

```
val commit : checkpoint_store -> checkpoint -> unit
```

```
val latest_checkpoint : checkpoint_store -> checkpoint option
```



```
val checkpoint_count : checkpoint_store -> int
```

```
type coordinator = {  
  workers : int;  
  store : checkpoint_store;  
  pending : worker_snapshot option array;  
  co_mu : Mutex.t;  
  co_done : Condition.t;  
}
```

```
val make_coordinator : workers:int -> store:checkpoint_store -> coordinator
```

```
val submit_snapshot : coordinator -> worker_snapshot -> unit
```

```
val wait_committed : coordinator -> epoch:epoch -> unit
```

```
val hash_key : string -> int -> int
```

```
type 'b transactional_sink = {  
  ts_write : epoch -> 'b -> unit;  
  ts_commit : epoch -> unit;  
  ts_abort : epoch -> unit;  
  ts_flush : unit -> unit;  
}
```

Sink с поддержкой two-phase commit, привязанного к checkpoint. Между barrier результаты пишутся в pre-commit (невидимы наружу); когда checkpoint зафиксирован — commit делает их видимыми; при сбое до commit — abort откатывает.

Это замыкает exactly-once на выходной стороне: переобработка после восстановления не создаёт дублей, потому что незакоммиченные результаты откатываются.

Для sink не умеющих транзакции используйте идемпотентный путь: `idempotent_sink` — детерминированный ключ + upsert.

```
val idempotent_sink : ('b -> unit) -> 'b0 transactional_sink
```

Идемпотентный sink: оборачивает обычную upsert-функцию. Commit/abort — no-op, потому что повторная запись по тому же детерминированному ключу безвредна (upsert перезапишет тем же). Самый дешёвый путь к корректному выходу, работает для любого sink с ключевой записью.

```
val buffered_sink : ('b list -> unit) -> 'b0 transactional_sink
```

Буферизующий транзакционный sink: накапливает результаты epoch, публикует пачкой на commit, отбрасывает на abort. Эмулирует 2PC для sink без нативных транзакций (in-memory pre-commit буфер).

```
val run_exactly_once :  
  workers:int ->  
  capacity:int ->  
  checkpoint_every:int ->  
  key_of:('a -> string) ->  
  make_state:(unit -> State_backend_memory.t) ->  
  process:(State_backend_memory.t -> 'a0 Mf_event.t -> 'b list) ->  
  source:'a1 seekable_source ->  
  sink:'b0 transactional_sink ->  
  store:checkpoint_store ->  
  unit ->
```

```
unit
```

`run_exactly_once` — data-parallel выполнение с полным end-to-end exactly-once:

- `source offset` фиксируется в каждом checkpoint (входная сторона);
- транзакционный `sink` коммитит результаты epoch только после того как checkpoint этого epoch зафиксирован (выходная сторона).

Параметры:

- `workers`, `capacity`, `checkpoint_every` — как раньше;
- `key_of` — ключ шардирования;
- `make_state` — создать backend воркера (вызывается N раз);
- `process backend epoch ev` — обработать событие, вернуть выходы;
- `source` — `seekable_source` (умеет position/seek);
- `sink` — `transactional_sink`;
- `store` — куда коммитить checkpoint-ы.

Воркер пишет выходы через `ts_write epoch`. Когда `barrier(epoch)` собран со всех воркеров и checkpoint зафиксирован, координатор вызывает `ts_commit epoch` — результаты становятся видимыми атомарно вместе с фиксацией стейта и offset.

```
val restore_latest :  
  checkpoint_store ->  
  State_backend_memory.t array ->  
  offset option
```

Раздать снимки последнего checkpoint воркерам. `backends.(i)` восстанавливается из snapshot воркера `i`. Возвращает offset с которого нужно перечитывать источник, либо `None` если checkpoint-ов нет.

```
val recover :  
  workers:int ->  
  make_state:(unit -> State_backend_memory.t) ->  
  source:'a seekable_source ->  
  checkpoint_store ->  
  State_backend_memory.t array
```

Протокол холодного старта после сбоя.

Единый путь восстановления: 1. прочитать последний валидный checkpoint из store; 2. восстановить стейт всех воркеров из снимков; 3. перемотать источник на сохранённый offset; 4. вернуть `backends` готовые к продолжению обработки.

Если checkpoint-ов нет (первый запуск) — `backends` пустые, источник остаётся в начале.

```
val serialize_checkpoint : checkpoint -> bytes
```

Сериализация checkpoint в bytes (Marshal). `worker_snapshot.state` уже bytes, поэтому Marshal записи безопасен.

```
val deserialize_checkpoint : bytes -> checkpoint
```

```
val durable_store : dir:string -> checkpoint_store
```

Создать store с durable-записью на диск. Каждый коммит пишет checkpoint в файл `dir/checkpoint_<epoch>.cp` плюс обновляет `dir/LATEST` с номером последнего epoch. Переживает рестарт процесса и (если `dir` на сетевом томе) смерть локального диска.

```
val load_durable : dir:string -> checkpoint_store
```

Загрузить последний durable checkpoint с диска в новый store (после рестарта процесса). Читает `dir/LATEST`, затем соответствующий `checkpoint_<epoch>.cp`.

Module Runtime

Запуск конвейера: точка входа в исполнение.

Связывает источник, пайплайн и sink, добавляя по режиму DLQ, метрики, graceful shutdown. Пользователь выбирает готовый `config` (`noop/log_cfg/parallel/prod`) или собирает свой.

```
type mode =  
  | Noop  
  | Log  
  | Parallel  
  | Prod
```

Режим работы.

```
type config = {  
  mode : mode;  
  parallelism : int;  
  capacity : int;  
  metrics_port : int;  
}  
  
число воркеров (для Parallel/Prod)  
  
ёмкость входных каналов воркеров  
  
порт Prometheus endpoint, 0 = выключено
```

Конфигурация запуска.

```
val noop : config
```

Без DLQ/метрик/shutdown — для тестов и простых прогонов.

```
val log_cfg : config
```

Метрики в stderr + graceful shutdown, один воркер.

```
val parallel : config
```

Параллельно (4 воркера), без HTTP-метрик.

```
val prod : config
```

Прод: параллельно + Prometheus endpoint на :9090.

```
val run :  
  ?label:string ->  
  config ->  
  key_of:( 'a -> string) ->  
  source:'a Mf_event.t Stream.t ->  
  pipeline:( 'a Mf_event.t Stream.t -> 'b Mf_event.t Stream.t) ->  
  sink:( 'b -> unit) ->  
  unit ->  
  unit
```

`run ?label cfg ~key_of ~source ~pipeline ~sink ()` исполняет `pipeline` над `source`, отправляя результаты в `sink`. `key_of` шардирует поток по воркерам в параллельных режимах. Сопутствующее (DLQ, метрики, обработчик

shutdown) включается согласно `cfg.mode. ?label` помечает метрики/логи этого пайплайна.

```
val safe_decode :  
  send_dlq:(Dlq_noop.entry -> unit) ->  
  topic:string ->  
  codec:(bytes -> ('a, string) Stdlib.result) ->  
  attempt:int ->  
  bytes ->  
  'a option
```

`safe_decode ~send_dlq ~topic ~codec ~attempt raw` декодирует `raw`: при успехе `Some v`, при ошибке отправляет запись в DLQ через `send_dlq` и возвращает `None` (битое сообщение не роняет пайплайн). Низкоуровневая деталь декодирования с DLQ; экспонирована для тестирования поведения «ошибка → DLQ, не падение».

```
val make_stream_with_dlq :  
  config ->  
  topic:string ->  
  codec:(bytes -> ('a, string) Stdlib.result) ->  
  ts_of:('a -> Time.t) ->  
  (unit -> (string * bytes) option) ->  
  'a Mf_event.t Stream.t
```

Module Harness

```
type ('a, 'b) t
```

Контекст теста: входная очередь + pipeline + выходной буфер

```
val create : ('a Mf_event.t Stream.t -> 'b Mf_event.t Stream.t) -> ('a, 'b) t
```

Создать контекст теста

```
val push : ('a, 'b) t -> 'a -> ts:int -> unit
```

Подать одно событие

```
val push_wm : ('a, 'b) t -> ts:int -> unit
```

Подать watermark

```
val push_all : ('a, 'b) t -> ('a * int) list -> unit
```

Подать список событий

```
val run : ('a, 'b) t -> 'b list
```

Закрыть входной поток и вычитать все выходные события

```
val run_data : ('a, 'b) t -> 'b list
```

Запустить и вернуть только Data значения

```
val run_all : ('a, 'b) t -> 'b Mf_event.t list
```

Запустить и вернуть все события включая Watermark/Retract

```
val expect_count : ('a, 'b) t -> int -> unit
```

Assertion helpers — бросают Failure с сообщением

```
val expect : ('a, 'b) t -> ('b -> bool) -> string -> unit
```

```
val expect_none : ('a, 'b) t -> unit
```

```
val ev : string -> int -> float -> float -> Domain.telemetry
```

Утилиты для построения тестовых событий

```
val evs : (string * int * float * float) list -> Domain.telemetry list
```

Module Barrier_default

```
type epoch = int
```

```
type coordinator = {  
  workers : int;  
  checkpoint_every : int;  
  mutable epoch : int;  
  ready : bool array;  
  mu : Mutex.t;  
  all_ready : Condition.t;  
}
```

```
val create : workers:int -> checkpoint_every:int -> coordinator
```

```
val worker_ready : coordinator -> worker:int -> epoch:'a -> unit
```

```
val wait_all_ready : coordinator -> epoch:'a -> unit
```

```
val current_epoch : coordinator -> int
```

```
val next_epoch : coordinator -> int
```

Module Barrier_noop

```
type epoch = int
```

```
type coordinator = {  
  mutable epoch : int;  
}
```

```
val create : workers:'a -> checkpoint_every:'b -> coordinator
```

```
val worker_ready : 'a -> worker:'b -> epoch:'c -> unit
```

```
val wait_all_ready : 'a -> epoch:'b -> unit
```

```
val current_epoch : coordinator -> int
```

```
val next_epoch : coordinator -> int
```


Module Config

Конфигурация как *типизированная запись*, не парсер файлов.

Библиотека определяет **какие** параметры существуют и их значения по умолчанию. **Откуда** их брать (YAML, JSON, env, CLI) — дело приложения: оно прочитает свой источник и соберёт эту запись. Так библиотека не тянет yaml-парсер на всех потребителей и не навязывает формат конфигурации.

Типичное использование приложением:

```
let cfg = { Config.default with workers = 8; checkpoint_every = 5000 }
```

или собрать из своих прочитанных значений и передать в runtime.

```
type t = {  
  workers : int;  
    число data-parallel воркеров  
  
  capacity : int;  
    ёмкость каждого входного канала  
  
  checkpoint_every : int;  
    barrier каждые N событий (0 = выключено)  
  
  watermark_latency_ms : int;  
    допуск на опоздание для watermark  
  
  watermark_interval_ms : int;  
    мин. шаг между эмиссией watermark (0 = на каждый)  
  
  state_dir : string;  
    каталог для durable checkpoint/RocksDB  
  
  metrics_interval_s : int;  
    период репорта метрик (0 = не репортить)  
}
```

```
val default : t
```

Значения по умолчанию — разумны для одиночного узла средней нагрузки.

```
val validate : t -> (t, string) Stdlib.result
```

Проверить корректность настроек. `Ok cfg` или `Error msg` с первым нарушением (например `workers <= 0`). Приложение зовёт это после сборки записи из своего источника, до запуска.

```
val to_json : t -> string
```

Сериализовать в JSON (для логирования эффективной конфигурации при старте — полезно видеть с чем реально запущен сервис).

Module D1q_log

```
type entry = D1q_noop.entry = {  
  topic : string;  
  payload : bytes;  
  error : string;  
  ts : int;  
  attempt : int;  
}
```

```
type t = {  
  mutable n : int;  
  mu : Mutex.t;  
}
```

```
val create : unit -> t
```

```
val send : t -> entry -> unit
```

```
val flush : 'a -> unit
```

```
val count : t -> int
```

Module D1q_noop

```
type t = {  
  mutable n : int;  
}
```

```
type entry = {  
  topic : string;  
  payload : bytes;  
  error : string;  
  ts : int;  
  attempt : int;  
}
```

```
val create : unit -> t
```

```
val send : t -> 'a -> unit
```

```
val flush : 'a -> unit
```

```
val count : t -> int
```

Module Fixtures

```
val ev : string -> int -> float -> float -> Domain.telemetry Mf_event.t
```

```
val scenario_alerts : Domain.telemetry Mf_event.t list
```

Module Health

Health/readiness как *структура*, не HTTP-сервер.

Библиотека вычисляет статус здоровья (значением), а поднять `/health` endpoint и выбрать HTTP-стек — дело приложения. Так библиотека не тянет `httpaf/dream` на всех потребителей и не навязывает решение про сеть.

Приложение периодически зовёт `check` и сериализует результат (`to_json`) в свой HTTP-ответ.

```
type readiness =  
  | Starting  
    ещё инициализируется  
  
  | Ready  
    принимает и обрабатывает нагрузку  
  
  | Draining  
    завершается (получен shutdown)  
  
  | Unhealthy  
    есть проблема (см. detail)
```

Готовность к работе.

```
type status = {  
  ready : readiness;  
  state_size : int;  
    число ключей в стеите (грубая оценка нагрузки)  
  
  watermark_lag_ms : int;  
    отставание watermark; большое = не успеваем  
  
  max_queue_depth : int;  
    макс. глубина очереди по воркерам  
  
  detail : string option;  
    пояснение для Unhealthy/Draining  
  
}
```

Снимок состояния.

```
val check :  
  ?readiness:(unit -> readiness) ->  
  ?state_size:(unit -> int) ->  
  ?watermark_lag_ms:(unit -> int) ->  
  ?max_queue_depth:(unit -> int) ->  
  unit ->  
  status
```

Источники данных для вычисления статуса. Приложение передаёт функции-замыкания на свои живые значения; `health` их опрашивает. Все опциональны — что не задано, считается нейтральным.

```
val to_json : status -> string
```

Сериализовать статус в JSON-строку (для HTTP-ответа приложения).

```
val is_live : status -> bool
```

Удобный предикат: считается ли статус «живым» для liveness-пробы (всё кроме `Unhealthy`).

Module Log

Структурированное логирование.

Библиотека порождает *структурированные* лог-события (уровень, сообщение, набор полей), а **куда** их писать — решает приложение через `set_sink`. По умолчанию события форматируются в JSON и пишутся в `stderr`. Приложение может перенаправить их в файл, Loki, свою систему — установив собственный sink, получающий уже готовую структуру `event`, а не строку.

Это держит границу библиотека/приложение чистой: библиотека решает *что* произошло (событие с полями), приложение — *куда* и *как* это пишется.

```
type level =  
  | Debug  
  | Info  
  | Warning  
  | Error
```

Уровень важности события.

```
type event = {  
  level : level;  
  message : string;  
  
  fields : (string * string) list;  
  
  ts_ms : int;  
}
```

человекочитаемое сообщение

структурированные пары ключ-значение

wall-clock метка, мс с эпохи

Структурированное лог-событие.

```
val to_json : event -> string
```

Сериализовать событие в одну строку JSON (поля `level`, `ts`, `msg` + все `fields`). Удобно для приложений пишущих в Loki/ELK.

```
val set_sink : (event -> unit) -> unit
```

Установить обработчик событий. По умолчанию — JSON в `stderr`. Приложение вызывает это чтобы перенаправить логи куда нужно.

```
val set_level : level -> unit
```

Минимальный уровень: события ниже отбрасываются (по умолчанию `Info`).

```
val emit : level -> ?fields:(string * string) list -> string -> unit
```

Породить событие. Обычно используются обёртки ниже.

```
val debug : ?fields:(string * string) list -> string -> unit
```

```
val info : ?fields:(string * string) list -> string -> unit
```

```
val warn : ?fields:(string * string) list -> string -> unit
```

```
val error : ?fields:(string * string) list -> string -> unit
```


Module Metrics_log

```
type counter = {  
  c_name : string;  
  c_labels : (string * string) list;  
  mutable c_val : int;  
  c_mu : Mutex.t;  
}
```

```
type gauge = {  
  g_name : string;  
  g_labels : (string * string) list;  
  mutable g_val : float;  
  g_mu : Mutex.t;  
}
```

```
type histogram = {  
  h_name : string;  
  h_labels : (string * string) list;  
  mutable h_count : int;  
  mutable h_sum : float;  
  mutable h_buckets : int array;  
  h_mu : Mutex.t;  
}
```

```
val buckets_us : int array
```

```
val all_counters : counter list Stdlib.ref
```

```
val all_gauges : gauge list Stdlib.ref
```

```
val all_histograms : histogram list Stdlib.ref
```

```
val reg_mu : Mutex.t
```

```
val pp_labels : (string * string) list -> string
```

```
val counter : name:string -> labels:(string * string) list -> counter
```

```
val gauge : name:string -> labels:(string * string) list -> gauge
```

```
val histogram : name:string -> labels:(string * string) list -> histogram
```

```
val incr : counter -> unit
```

```
val add : counter -> int -> unit
```

```
val set_gauge : gauge -> float -> unit
```

```
val observe : histogram -> float -> unit
```

```
val dump : unit -> string
```

```
val start_reporter : interval_s:int -> unit
```

```
val start_server : ?port:int -> unit -> unit
```

Module Metrics_noop

```
type counter = unit
```

```
type gauge = unit
```

```
type histogram = unit
```

```
val counter : name:'a -> labels:'b -> unit
```

```
val gauge : name:'a -> labels:'b -> unit
```

```
val histogram : name:'a -> labels:'b -> unit
```

```
val incr : 'a -> unit
```

```
val add : 'a -> 'b -> unit
```

```
val set_gauge : 'a -> 'b -> unit
```

```
val observe : 'a -> 'b -> unit
```

```
val dump : unit -> string
```

```
val start_reporter : interval_s:'a -> unit
```

Module Retry

Политика повторных попыток с экспоненциальной задержкой.

Оборачивает операцию, которая может временно сбойть (decode, обработка, запись в sink). При ошибке повторяет с растущей паузой; при исчерпании попыток вызывает финальный обработчик (обычно — отправку в DLQ). Это слой *над* DLQ: DLQ — конечная остановка, retry — попытки до неё.

Backoff: задержка попытки $k = \text{base_ms} * \text{factor}^{(k-1)}$, но не больше max_ms . Опциональный jitter сглаживает «громовое стадо» при массовых одновременных сбоях.

```
type policy = {
  max_attempts : int;
  

максимум попыток (>= 1)



  base_ms : int;
  

базовая задержка перед 2-й попыткой



  factor : float;
  

множитель роста (обычно 2.0)



  max_ms : int;
  

потолок задержки



  jitter : bool;
  

добавлять случайный разброс 0..delay)


}
```

```
val default : policy
```

Разумная политика по умолчанию: 5 попыток, 100мс база, *2, потолок 5с.

```
val delay_for : policy -> int -> int
```

Задержка (мс) перед попыткой номер `attempt` (1-based; для attempt=1 всегда 0 — первая попытка без паузы). Чистая функция, тестируемая.

```
val with_retry :
  policy ->
  sleep:(int -> unit) ->
  on_give_up:(exn -> int -> unit) ->
  ('a -> 'b) ->
  'a ->
  'b option
```

`with_retry policy ~sleep ~on_give_up f x` выполняет `f x`. Если `f` бросает исключение — повторяет по политике, делая паузу через `~sleep` (мс). Если все попытки исчерпаны — зовёт `~on_give_up` с последним исключением и числом попыток, и возвращает `None`. При успехе — `Some result`.

`~sleep` вынесен параметром чтобы тесты не ждали реального времени (передают `no-op`), а прод передаёт настоящий сон.

Module Rocksdb_ffi

```
type db
```

```
val open_db : string -> db
```

```
val close_db : db -> unit
```

```
val put : db -> string -> bytes -> unit
```

```
val get : db -> string -> bytes option
```

```
val delete : db -> string -> unit
```

Module Schema_default

```
type version = int
```

```
type 'a t = {  
  enc : 'a -> bytes;  
  dec : bytes -> ('a, string) Stdlib.result;  
  ver : version;  
  migrate : (version -> bytes -> bytes) option;  
}
```

```
val make :  
  version:version ->  
  encode:('a -> bytes) ->  
  decode:(bytes -> ('b, string) Stdlib.result) ->  
  ?migrate:(version -> bytes -> bytes) ->  
  unit ->  
  'a t
```

```
val encode : 'a t -> 'b -> bytes
```

```
val decode : 'a t -> bytes -> ('b, string) Stdlib.result
```

```
val current_version : 'a t -> version
```

Module Schema_noop

```
type version = int
```

```
type 'a t = {  
  enc : 'a -> bytes;  
  dec : bytes -> ('a, string) Stdlib.result;  
  ver : version;  
}
```

```
val make :  
  version:version ->  
  encode:('a -> bytes) ->  
  decode:(bytes -> ('b, string) Stdlib.result) ->  
  ?migrate:'c ->  
  unit ->  
  'a t
```

```
val encode : 'a t -> 'b -> bytes
```

```
val decode : 'a t -> bytes -> ('a, string) Stdlib.result
```

```
val current_version : 'a t -> version
```

Module Shutdown_default

```
val requested : bool Stdlib.ref
```

```
val mu : Mutex.t
```

```
val cond : Condition.t
```

```
val request : unit -> unit
```

```
val is_requested : unit -> bool
```

```
val register : on_shutdown:(unit -> unit) -> unit
```

```
val wait : unit -> unit
```


Module Shutdown_noop

```
val register : on_shutdown:'a -> unit
```

```
val is_requested : unit -> bool
```

```
val wait : unit -> unit
```

```
val request : unit -> unit
```

Module State_backend_memory

```
type t = (string, bytes) Stdlib.Hashtbl.t
```

```
val create : unit -> ('a, 'b) Stdlib.Hashtbl.t
```

```
val get : ('a, 'b) Stdlib.Hashtbl.t -> 'c -> 'b option
```

```
val set : ('a, 'b) Stdlib.Hashtbl.t -> 'c -> 'd -> unit
```

```
val delete : ('a, 'b) Stdlib.Hashtbl.t -> 'c -> unit
```

```
val keys : ('a, 'b) Stdlib.Hashtbl.t -> 'c list
```

```
val size : ('a, 'b) Stdlib.Hashtbl.t -> int
```

```
val snapshot : ('a, 'b) Stdlib.Hashtbl.t -> bytes
```

```
val restore : (string, bytes) Stdlib.Hashtbl.t -> bytes -> unit
```

Module State_backend_noop

```
type t = unit
```

```
val create : unit -> unit
```

```
val get : 'a -> 'b -> 'c option
```

```
val set : 'a -> 'b -> 'c -> unit
```

```
val delete : 'a -> 'b -> unit
```

```
val keys : 'a -> 'b list
```

```
val size : 'a -> int
```

```
val snapshot : 'a -> bytes
```

```
val restore : 'a -> 'b -> unit
```

Module State_backend_rocksdb

```
type t = {  
  db : Rocksdb_ffi.db;  
  path : string;  
  mutable known : (string, unit) Stdlib.Hashtbl.t;  
}
```

```
val default_dir : string
```

```
val create_at : string -> t
```

```
val create : unit -> t
```

```
val get : t -> string -> bytes option
```

```
val set : t -> string -> bytes -> unit
```

```
val delete : t -> string -> unit
```

```
val keys : t -> string list
```

```
val size : t -> int
```

```
val close : t -> unit
```

```
val snapshot : t -> bytes
```

```
val restore : t -> bytes -> unit
```